National University of Singapore
School of Computing

Semester 1, AY2026-27

CS4246/CS5446 Reinforcement Learning and Sequential Decision Making

---

Issued: 14 October 2026 **Due: Friday 30 October 2026, 23:59**

Rush Hour Adventures: Episode II - Reinforcement Learning

Guidelines

Please complete the assignment in your existing tutorial teams as formed on Canvas. Only ONE member per team should submit all answers. Grading will be done on a team basis.

On collaboration

You are encouraged to discuss solution ideas. However, each team *must write its solutions independently*. Highly similar solutions from different teams or other sources are considered plagiarism. At the top of both completed notebooks, include a Markdown cell listing the names and matriculation numbers of both team members, any collaborators (write **None** if there are none), and any sources used.

Getting started on Google Colab

- Download the programming package from Canvas or from this GitHub repo.
- Upload the *entire* `CSXX46A2-Student` folder to the top level of Google Drive (`My Drive`). Keep the notebooks, `utils.py`, `MyWrappers.py`, and `selfDefinedEnvs` together in that folder.
- Open each template notebook from that uploaded folder in Google Colab. The supplied setup cells expect `/content/drive/MyDrive/CSXX46A2-Student`; if you store the folder elsewhere, update the path in each notebook before running it.
- In each Colab notebook, choose `Runtime` → `Change runtime type` → `GPU` before training. Run the setup cells and confirm that the intended assignment folder and GPU are detected. Save the notebooks back to Drive after training and evaluation so their cell outputs are retained.

Assignment submission

- Complete the marked task blocks in both template notebooks. Preserve the supplied function names, interfaces, and support-file layout so that the trained checkpoints can be loaded by the grader.

- Submit the **completed template notebooks with all cells run**, including the training diagnostic graphs and final evaluation outputs. Save each notebook after its run; notebooks with cleared outputs are not sufficient.
- Save all four trained checkpoints using the exact filenames listed below. Submit the checkpoints produced by your own completed notebooks, not the legacy checkpoints.
- Create one ZIP file named `groupXX_assignment2.zip`, replacing `XX` with your registered Canvas group number. Put **exactly the following six files at the ZIP root**, without an enclosing directory or a second ZIP:
  - `Model-FreeRL-Template.ipynb`
  - `Model-BasedRL-Template.ipynb`
  - `model_free_agent.pth`
  - `model_based_agent.pth`
  - `transition_model.pth`
  - `reward_model.pth`
- **Only one group member submits this ZIP to Canvas.** Before submitting, reopen the ZIP and verify the six filenames, the saved notebook outputs, and the presence of all four nonempty checkpoints. The grader will load the checkpoints with its own trusted environment and support files; it will not rely on arbitrary replacement support code inside the submission.

Background: The Elevator Domain

Welcome to the Elevator Challenge, episode II! Your mission, should you choose to accept it, is to design and implement efficient learning solutions to manage an elevator that serves impatient office workers who want to reach the ground floor. This seemingly simple scenario presents a large state space, stochastic passenger arrivals, and long-term consequences for each action.

We study the elevator domain from Assignment 1. The building has 5 floors and 1 elevator. Each floor can have at most 3 people waiting, and the elevator has a capacity of 10 people. The objective is to collect passengers from the upper floors and deliver them to the ground floor. New passengers may arrive at every time step. The elevator can move in its current direction, open its door, or close its door. Passengers board on upper floors only while the door is open, delivery at the ground floor is automatic, and the elevator can move only while the door is closed.

The state space is too large for the tabular methods used earlier in the course. In Problem 1, you implement model-free Proximal Policy Optimization (PPO). In Problem 2, you extend PPO to a model-based setting by learning transition and reward models and using them to generate imagined rollouts.

To understand the domain, explore the provided `ElevatorEnv.ipynb`, which describes the state space, action space, and environment interaction. The supplied `selfDefinedEnvs` folder contains the RDDL domain and instances used by both problems.

Instructions on Solution

The assignment can be completed in Jupyter, and we encourage you to use Google Colab for access to GPU resources. Detailed instructions are provided in the following notebooks:

- Problem 1: `Model-FreeRL-Template.ipynb`
- Problem 2: `MBRL-Template.ipynb`

Keep each notebook, `utils.py`, and the `selfDefinedEnvs` folder together. All blocks that you must complete are indicated by comments of the following form, where `N` identifies a task:

```
### ------------- TASK N ------------- ###
### ----- YOUR CODES START HERE ------ ###
### ------ YOUR CODES END HERE ------- ###
```

Prerequisites: NumPy and PyTorch (Optional)

If you are unfamiliar with NumPy and PyTorch for machine learning, you may refer to CS285 Lecture 3 and the corresponding CS285 Lecture 3 Google Colab notebook for a technical and hands-on tutorial.

Problem 1: Proximal Policy Optimization (PPO)

In this problem, you will implement Proximal Policy Optimization (PPO) to solve the Elevator task. PPO is an actor–critic algorithm that uses a clipped surrogate objective to prevent a policy update from moving too far from the behavior policy used to collect the rollout [1].

[1] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347. https://arxiv.org/abs/1707.06347.

PPO maintains an actor and a critic. For a state s, the actor defines a categorical policy $\pi_\theta(a \mid s)$ over the discrete actions, and the critic estimates the state value $V_\phi(s)$. Here, θ and ϕ denote the actor and critic parameters, respectively.

There are three groups of tasks in Problem 1:

1. Complete the actor–critic network and its forward functions.
2. Given rollout data, compute the temporal-difference residual used by generalized advantage estimation.
3. Implement the clipped policy objective, clipped value loss, and combined actor–critic loss.

For further instructions, follow `Model-FreeRL-Template.ipynb`.

Problem 2: Model-Based Proximal Policy Optimization (MB-PPO)

Let s_t be the 13-dimensional elevator state, a_t the discrete action, r_t the reward, s_{t+1} the next state, and d_t the episode-end indicator. In the model-based extension, the agent stores real transitions $(s_t, a_t, r_t, s_{t+1}, d_t)$ in a replay buffer. These transitions train a transition model $\widehat{P}_\psi(s_{t+1} \mid s_t, a_t)$ and a reward model $\widehat{r}_\omega(s_t)$. After a pretraining phase, the learned models generate fixed-length imagined rollouts, and PPO updates the actor and critic using those rollouts.

The transition model respects the structure of the elevator state. The number of people waiting on each of the four upper floors is stochastic and is represented by a four-class categorical head. The number of people in the elevator uses 11 classes, corresponding to counts 0 through 10. The direction and door components use binary heads, and the elevator location uses a five-class head whose prediction is converted to a one-hot vector. The ground-floor waiting count is always zero.

There are seven model-learning and model-based training tasks:

1. Define the eight prediction heads of the learned transition model.
2. Concatenate the state and action encodings and complete the transition model's forward pass.
3. Add valid transitions collected from the real vectorized environments to the replay buffer.
4. Start learned-model training only after the replay buffer contains a complete model mini-batch.

5. Compute a cross-entropy loss for every transition-model output. The supplied reward-model loss remains a mean-squared error.
6. Start PPO training in the learned model only after the model-pretraining phase has ended.
7. Construct each predicted next state by sampling the stochastic counts, taking the most likely deterministic classes, and converting the predicted elevator floor to one-hot form.

Problem 2 also contains PPO helper blocks labelled PPO Tasks 1.1, 1.2, 2, 3.1.1, 3.1.2, 3.2, and 3.3. Complete these by adapting your corresponding actor, log-probability, advantage, policy-objective, value-loss, and total-loss implementations from Problem 1.

Finally, complete the evaluation interaction loop, report the average undiscounted return in the real environment, and report the learned transition model's one-step prediction error. An episode ends when either `terminated` or `truncated` is true.

For further instructions and public sanity checks, follow `Model-BasedRL-Template.ipynb`.

Good luck, and have fun exploring model-free and model-based reinforcement learning!